1. **Задание**

1. Изучите заданную функцию:

$$f(x) = (x - a)^2 + b \cdot \sin cx$$

где a, b и c – параметры, зависящие от варианта.

2. Построение графика функции:
   - Используйте библиотеку matplotlib для построения графика функции
   $$f(x) = (x - a)^2 + b \cdot \sin cx.$$
   - Отметьте на графике начальный отрезок $[x_0, x_1]$.
3. Реализация методов поиска минимума:
   - Выберите два из трех методов. Напишите функции для метода золотого сечения, дихотомии или метода параболической интерполяции.
   - Визуализируйте процесс поиска минимума, отображая на графике точки, выбираемые на каждом шаге алгоритма.
4. Сравнение методов:
   - Постройте графики сходимости для обоих методов.
   - Сравните количество итераций, необходимое для достижения заданной точности.

2. **Вариант**

Согласно варианту №6

№ варианта	Параметр a	Параметр b	Параметр c	Начальный отрезок $[x_0, x_1]$	Точность ε
6	0.3	2.0	1.3	[-2.5, 2.5]	0.0001

Для сравнения выбраны методы золотого сечения и дихотомии

4. **Графики функций с итерационным поиском**

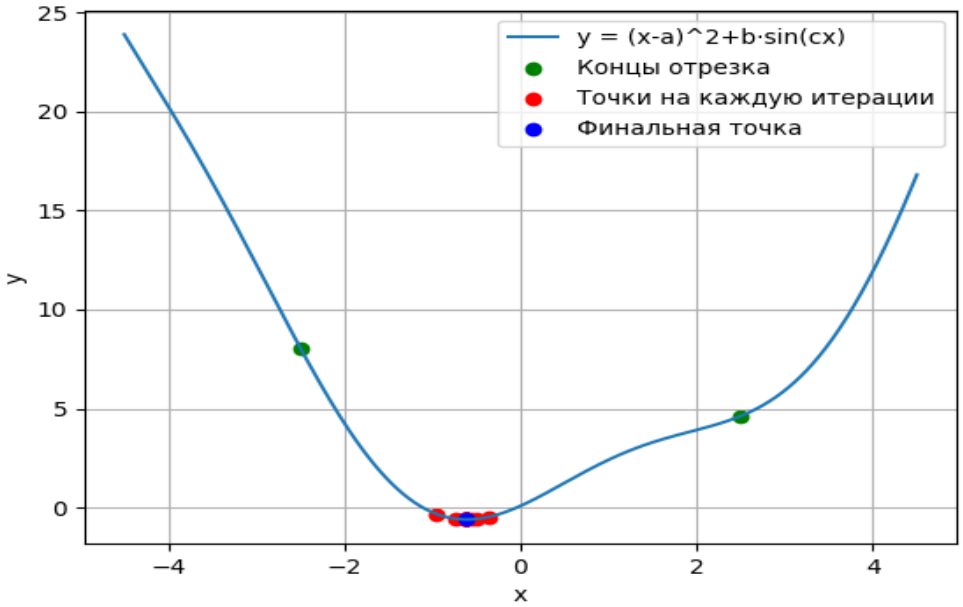


Рисунок 1 – Поиск с помощью золотого сечения

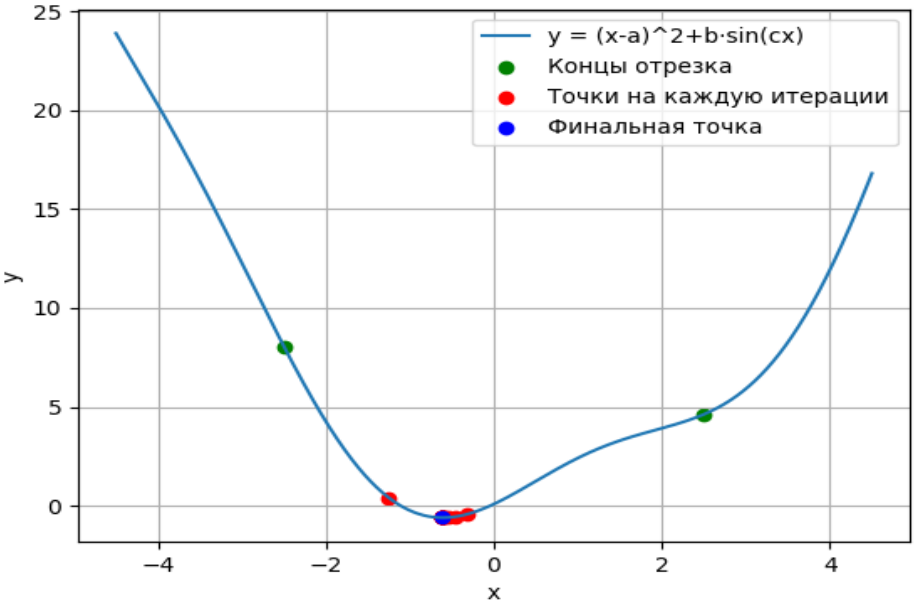


Рисунок 2 – Поиск с помощью дихотомии

5. **Вычисления**

   Метод золотого сечения нашёл минимум в точке -0.6110071950015383 со значением минимума -0.5968254616522305 за 23 итераций

   Метод дихотомии нашёл минимум в точке -0.6110088821293751 со значением минимума -0.5968254616585023 за 30 итераций

6. **Графики сходимости**

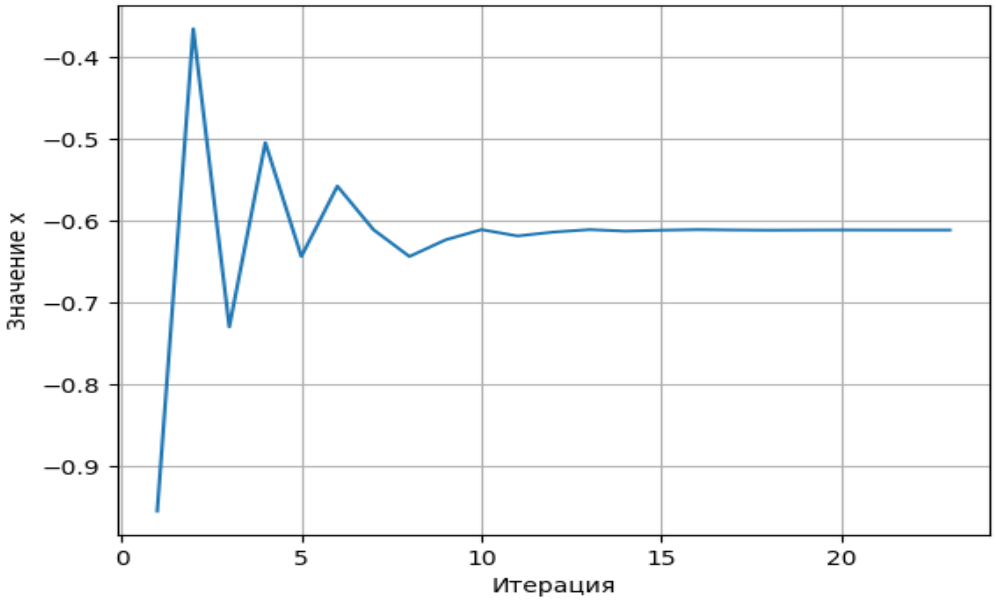


Рисунок 3 – График сходимости метода золотого сечения

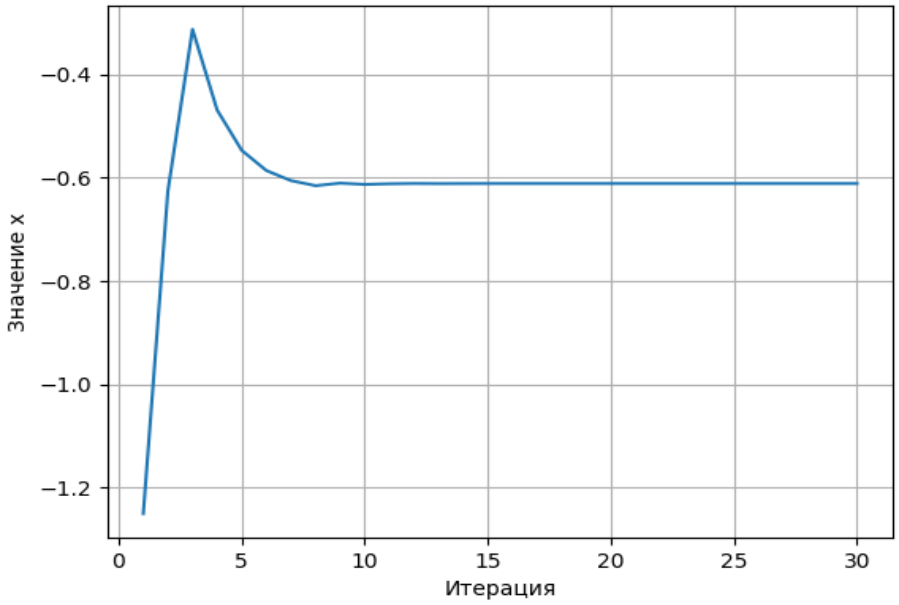


Рисунок 4 – График сходимости метода дихотомии

7. Выводы

На основе результатов, полученных после выполнения лабораторной работы, можно сделать вывод, что при заданной точности, метод золотого сечения позже подходит к истинному значению в начале, но сохраняет скорость приближения, в отличие от метода дихотомии, который быстрее подходит к истинному значению в начале, но сбавляет скорость ближе к концу.

8. Листинг программного кода

```
import numpy as np
import matplotlib.pyplot as plt

# Вариант 6
# Золотое сечение и дихотомия

Var_a = 0.3
Var_b = 2
Var_c = 1.3

Var_x_min = -2.5
Var_x_max = 2.5
Epsilon = 0.0001

# Вспомогательные методы

# Функция по заданию
def function(x):
    return (x-Var_a)**2 + Var_b * np.sin(Var_c * x)

# Функция поиска минимума по золотому сечению
def method_golden_section(f, a, b, epsilon):
    r = (np.sqrt(5) - 1) / 2  # ≈ 0.618

    x1 = a + (1 - r) * (b - a)
    x2 = a + r * (b - a)
    f1 = f(x1)
    f2 = f(x2)

    x_mins = []
    while abs(b - a) > epsilon:
        if f1 > f2:
            a = x1
            x1 = x2
```

```
            f1 = f2
            x2 = a + r * (b - a)
            f2 = f(x2)
        else:
            b = x2
            x2 = x1
            f2 = f1
            x1 = a + (1 - r) * (b - a)
            f1 = f(x1)
        x_mins.append((a + b) / 2)

    return x_mins, (a + b) / 2

# Функция поиска минимума по методу дихотомии
def method_dichotomy(f, a, b, epsilon):
    delta = epsilon / (2+epsilon)

    x_mins = []
    while abs(b - a) > epsilon:

        x1 = (a + b) / 2 - delta
        x2 = (a + b) / 2 + delta

        if f(x1) < f(x2):
            b = x2
        else:
            a = x1
        x_mins.append((a + b) / 2)

    return x_mins, (a + b) / 2

# Основной алгоритм #

# Константа для расширения диапазона
x_constant = 2
```

```
# Отрезок точек для функции
x_points = np.linspace(Var_x_min - x_constant, Var_x_max + x_constant, num=100000)
y_points = function(x_points)

# Расчёт минимума метода золотого сечения и дихотомии
golden_x, x_golden_min = method_golden_section(function, Var_x_min, Var_x_max, Epsilon)
dichotomy_x, x_dichotomy_min = method_dichotomy(function, Var_x_min, Var_x_max,
Epsilon)

# Отрисовка золотого сечения
fig, ax = plt.subplots()
fig.canvas.manager.set_window_title("График поиска минимума по золотому сечению")

ax.plot(x_points, y_points, label="y = (x-a)^2+b·sin(cx)")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.grid(True)

# Отрисовка границ отрезка
x_end_of_function = np.array([Var_x_min, Var_x_max])
y_end_of_function = function(x_end_of_function)
ax.scatter(x_end_of_function, y_end_of_function, color="green", label="Концы отрезка")
ax.legend()

# Отрисовка границ отрезка
golden_x_without_last = golden_x[:-1]
y_golden = function(np.array(golden_x_without_last))
ax.scatter(golden_x_without_last, y_golden, color="red", label="Точки на каждую итерации")
ax.scatter(x_golden_min, function(x_golden_min), color="blue", label="Финальная точка")
ax.legend()

# Отрисовка дихотомии
fig, ax = plt.subplots()
fig.canvas.manager.set_window_title("График поиска минимума по дихотомии")
```

```
ax.plot(x_points, y_points, label="y = (x-a)^2+b·sin(cx)")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.grid(True)

# Отрисовка границ отрезка
x_end_of_function = np.array([Var_x_min, Var_x_max])
y_end_of_function = function(x_end_of_function)
ax.scatter(x_end_of_function, y_end_of_function, color="green", label="Концы отрезка")
ax.legend()

# Отрисовка границ отрезка
dichotomy_x_without_last = dichotomy_x[:-1]
y_dichotomy = function(np.array(dichotomy_x_without_last))
ax.scatter(dichotomy_x_without_last, y_dichotomy, color="red", label="Точки на каждую итерации")
ax.scatter(x_dichotomy_min, function(x_dichotomy_min), color="blue", label="Финальная точка")
ax.legend()

# График сходимости золотого сечения
fig, ax = plt.subplots()
fig.canvas.manager.set_window_title("График сходимости золотого сечения")
iterations = np.arange(1, len(golden_x)+1)
print(x_golden_min, function(x_golden_min), len(iterations))

# График приближений
plt.plot(iterations, golden_x)

plt.xlabel("Итерация")
plt.ylabel("Значение x")
plt.grid(True)
# График сходимости дихотомии
fig, ax = plt.subplots()
```

```
fig.canvas.manager.set_window_title("График сходимости метода дихотомии")
iterations = np.arange(1, len(dichotomy_x)+1)
print(x_dichotomy_min, function(x_dichotomy_min), len(iterations))

# График приближений
plt.plot(iterations, dichotomy_x)

plt.xlabel("Итерация")
plt.ylabel("Значение x")
plt.grid(True)

# Отрисовка всех графиков
plt.show()
```